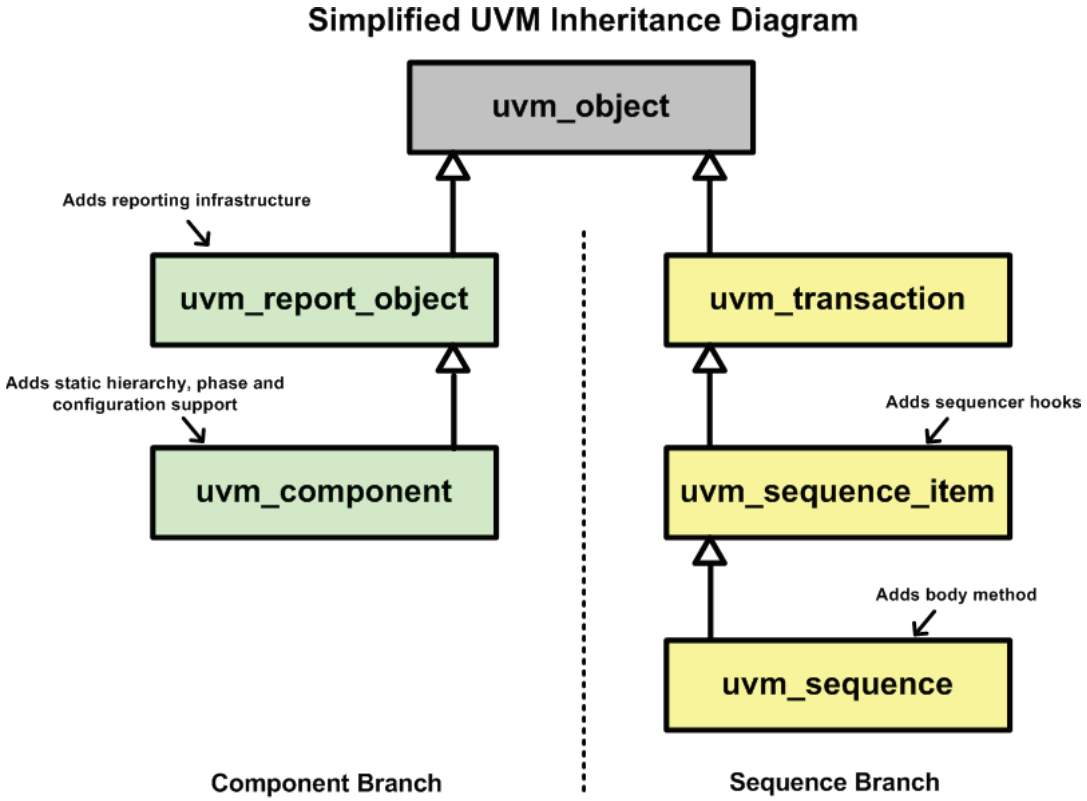
The UVM Messaging System

UVM Messaging

The UVM messaging system provides an infrastructure for printing messages in a consistent format from a UVM testbench. The messaging service is built into all UVM components, since they are extended from the uvm_report_handler as shown in the UVM class inheritance diagram. Messaging can also be used from UVM objects such as sequence_items and sequences.



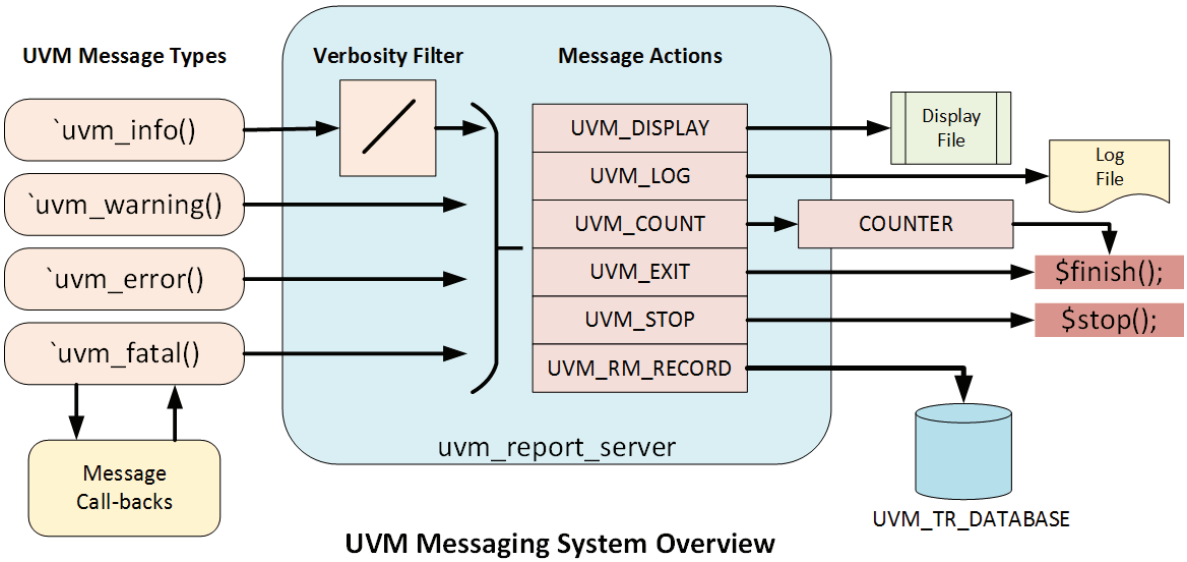
There are four message types supported by the UVM messaging system:

Message Type	Suggested useage
UVM_INFO	Should be used for informative messages. Its verbosity can be controlled
UVM_WARNING	Should be used for warnings to the user
UVM_ERROR	Should be used to flag errors to the user
UVM_FATAL	Should be used for errors that will prevent the test from doing anything useful. Also results in an exit from the simulation

The system has a verbosity control capability that allows the suppression of UVM_INFO messages. There is no verbosity control for UVM_WARNING, UVM_ERROR and UVM_FATAL messages.

UVM messages can have actions associated with them, and these allow control of where the message is sent; whether the message causes the simulation to stop or finish; or whether the message is stored in the UVM transaction database. For more detail about the features of the messaging system, see the section on using UVM messaging.

In addition to UVM based messaging API calls, the verbosity of the message reporting system can also be controlled, to a lesser extent, from the simulation command line.



UVM Messaging System Overview

A message can be intercepted by a report catcher call-back, allowing its content, type and severity to be changed. This feature is useful in some circumstances, for instance to demote an error to a warning, but should be used with care.

At the end of the simulation, the message server can be interrogated to see how many messages of a particular type have been issued. This is particularly useful as a sanity check in regressions to ensure that no error or warning messages have been reported.

When messages are sent from a UVM component, the message includes the UVM testbench path to the component. When messages are sent from objects such as sequences, then the path part of the message uses the result of the get_full_name() method. For sequences this does not include the full sequence hierarchy unless you follow a particular approach to starting sequences. See the article on sequence messaging for more details.

Printing a message has an overhead associated with it since there will be string processing in the UVM library and there will be some form of File I/O overhead in the simulator. This means that a noisy testbench can also be a slow testbench, so it is worth considering how verbose your messaging output needs to bewhen you are coding a verification component or a sequence.

Using Messaging

Messaging macros

The recommended way to use the UVM messaging system is to use the message macros, since they automatically insert the file name and line number of the message source into the UVM message string which is useful for debugging. In the case of information messages, the expanded macro code also checks the verbosity setting for the message to determine whether it should be printed before any string processing takes place. This can improve testbench performance.

The macros are:

```
`uvm_fatal(“message_id”, “message_string”)
`uvm_error(“message_id”, “message_string”)
`uvm_warning(“message_id”, “message_string”)
`uvm_info(“message_id”, “message_string”, uvm_verbosity)
```

Where:

The **message_id** is a string that can be used to identify the source of the message. It is used within the messaging system as a reference that allows you to control other aspects of the message behavior.

The **message_string** is what will be printed in the body of the message, it can be a simple string or it can be the result of a string formatting function such as $sformatf().

The **uvm_verbosity** is an enumerated type that defines the verbosity setting for the message.

Message Verbosity

The UVM has five levels of verbosity which are defined as an enum within the UVM package:

Verbosity Level	Effect on message
UVM_NONE	A message with this level of verbosity will always be printed regardless of the verbosity setting
UVM_LOW	A message with this level of verbosity will be printed if the verbosity is set to UVM_LOW, UVM_MEDIUM, UVM_HIGH or UVM_FULL
UVM_MEDIUM	A message with this level of verbosity will be printed is the verbosity is set to UVM_MEDIUM, UVM_HIGH or UVM_FULL. This is the default level.
UVM_HIGH	A message with this level of verbosity will be printed if the verbosity is set to UVM_HIGH or UVM_FULL
UVM_FULL	A message with this level of verbosity will be printed if the verbosity is set to UVM_FULL

The verbosity settings are a common source of confusion amongst users. The important thing to realize is that the name of the enum for a particular verbosity setting is related to the amount of verbosity allowed within a testbench or part of a testbench. The idea being that FULL relates to all possible messages being printed, and NONE to only essential messages being printed. For example, if an info message has a verbosity setting of UVM_HIGH, it will only be emitted if the verbosity setting is set to UVM_HIGH or UVM_FULL.

Each info message has to have a verbosity setting. Whether it is displayed or not is determined by the verbosity setting of the UVM testbench or the component from which it is generated.

Suggested uses of the settings are shown in the table below:

Verbosity Level	Suggested Usage
UVM_NONE	A message that will always be printed – e.g. Mandatory information such as Copyright or design/testbench configuration.
UVM_LOW	A message that should normally be printed, but could be filtered by a UVM_NONE setting. – e.g. Higher level messaging, Assertion messaging
UVM_MEDIUM	Debug level 1 – e.g. Phase has started/finished (Default verbosity level)
UVM_HIGH	Debug level 2 – e.g. Detailed transaction messages
UVM_FULL	Low level of detail debug messages

The UVM messaging verbosity is set to UVM_MEDIUM by default. The message verbosity level can be changed either for the whole UVM testbench or at a finer level of granularity.

Message IDs

The message ID field of a UVM message can be used as a way of grouping related messages and then controlling how they are handled. The ID string can take any format, and there is no limit as to how many can be used.

Message Actions

UVM_ACTION_e	Description
UVM_NONE	No action is taken – an alternative way to suppress a message.
UVM_DISPLAY	The message is directed to the simulation transcript
UVM_LOG	The message is directed to a specific log file
UVM_COUNT	The message increments an exit counter value
UVM_EXIT	The message causes the simulation to finish immediately
UVM_STOP	The message causes the simulation to stop and become interactive
UVM_RM_RECORD	The message is logged in the uvm_tr_database

These actions are encoded as a bit pattern and can be ORed together to create a hybrid action definition.

For any component, the actions for different message types can be changed either according to their severity or their id, or both. The various message action API calls are:

```
// Apply to a single level of the component hierarchy:
set_report_severity_action(uvm_severity severity, uvm_action action);
set_report_id_action(string id, uvm_action action);
set_report_severity_id_action(uvm_severity, string id, uvm_action
action);

// For instance:
// Any `uvm_info() messages from this_component with an id of
"this_agent" are sent to a log file and the transcript
this_component.set_report_severity_id_action(UVM_INFO, "this_agent",
UVM_LOG | UVM_DISPLAY);

// Apply to all components below the component in the hierarchy:
set_report_severity_action_hier(uvm_severity severity, uvm_action
action);
```

```
set_report_id_action_hier(string id, uvm_action action);
set_report_severity_id_action_hier(uvm_severity, string id, uvm_action
action);
```

Using UVM_LOG

In order to use the UVM_LOG action, you need to take responsibility for opening the log file before any messages are written to it, and for closing it at the end of the simulation. You also need to pass the file handle to the messaging system. Again, the log file can be set up as a default, or according to severity, id or both. The API calls are:

```
// Apply to a single level of the component hierarchy:
set_report_default_file(UVM_FILE file);
set_report_id_file(string id, UVM_FILE file);
set_report_severity_file(uvm_severity severity, UVM_FILE file);
set_report_severity_id_file(uvm_severity severity, string id, UVM_FILE
file);

// Apply to all components below the component in the hierarchy:
set_report_default_file_hier(UVM_FILE file);
set_report_id_file_hier(string id, UVM_FILE file);
set_report_severity_file_hier(uvm_severity severity, UVM_FILE file);
set_report_severity_id_file_hier(uvm_severity severity, string id,
UVM_FILE file);
```

The following code is an example of how the log file handle is created, applied and then how the log file is closed at the end of the simulation:

```
task run_phase(uvm_phase phase);

  // Create a file handle by opening a file:
  UVM_FILE green_log_fh = $fopen("green_messages.log");

  // Adding the log action for the "green_id":
  env.green.set_report_id_action("green_id", (UVM_DISPLAY | UVM_LOG));
  // Setting the file handle for the log action
  env.green.set_report_id_file("green_id", green_log_fh);


  phase.raise_objection(this);
  #1us;
  // Test code goes here
  phase.drop_objection(this);

  // Closing the log file at the end of test:
  $fclose(green_log_fh);
endtask
```

Note that an alternative place to close the file might be at the end of the report_phase(), especially if the report_phase() is going to write to the file!

Using UVM_COUNT

The UVM_COUNT action is useful for controlling how many errors occur before the UVM test is aborted, but in principle it can be used to control how many messages of any type allocated the UVM_COUNT action occur before the simulation exits. The maximum value of the counter is set using the set_max_quit_count() API call.

Using UVM_EXIT/UVM_STOP

The UVM_EXIT action means that if a message with this action is called, the simulation stops immediately. The uvm_fatal message class uses this action by default.

The UVM_STOP action makes the simulation stop and puts the simulator into interactive mode. This could be useful for debug.

Using UVM_RM_RECORD

The UVM_RM_RECORD action adds an entry to the UVM transaction database when the message is generated. This is a new option in the 1800.2 version of the UVM. How the transaction database works and how it is used is specific to a vendor implementation.

Default Message Action Settings

The four types of UVM message have default action settings as follows:

Message Type	Default Action Settings
UVM_INFO	UVM_DISPLAY
UVM_WARNING	UVM_DISPLAY
UVM_ERROR	UVM_DISPLAY|UVM_COUNT
UVM_FATAL	UVM_DISPLAY|UVM_EXIT

UVM Report Catcher

There are situations where you may need to change a message generated by the messaging system, and the uvm_report_catcher is built-in call-back mechanism for doing this. Typical applications might be to downgrade an error to a warning, or to modify a message. The report catcher can modify the severity, verbosity, id, action or the string content of a message before it is passed to the message server for printing.

The report catcher is implemented as an extended version of the uvm_callbacks object. Multiple report catchers can be registered and potentially each message will be processed by all the registered call-backs in the order in which they were registered. The report catcher's catch() method is called as the message is generated, and the implementation of the catch() method determines how the message is processed. If the catch() method returns a THROW value, then the message is passed onto other registered report catchers. If it returns a CAUGHT value then the message is passed to the report server. Inside the catch() method, the issue() call can be made which will immediately send the message to the report server.

Each uvm_report_catcher object needs to be constructed and then registered either as a global report catcher or as a report catcher for a group of components or a single component.

Report Catcher Examples

The first report catcher example defines a call-back that checks for a message with the "green_id" and if the message type is UVM_INFO, then it modifies the message. The same call-back also demotes any errors with a "green_id" to warnings. The call back returns a THROW value, which means that the message will be passed to the next report catcher call-back, if there is one.

```
// Example report catchers:
class message_mod extends uvm_report_catcher;
`uvm_object_utils(message_mod)

function new(string name = "message_mod");
  super.new(name);
endfunction

function action_e catch();
  string message;
  string id;

  if((get_id() == "green_id") & (get_severity() == UVM_INFO)) begin
    set_message("Message modified");
  end
  else if((get_id() == "green_id") & (get_severity() == UVM_ERROR))
begin
    set_severity(UVM_WARNING);
    set_message("This warning was an error");
  end

  return THROW;

endfunction
```

```
endclass
```

The following report catcher example is designed to catch a UVM_FATAL severity message with a "red_id" and demote it to an error. This type of report catcher might be useful in the early stages of debugging a testbench as a means of continuing past a fatal error.

```
class fatal_mod extends uvm_report_catcher;
`uvm_object_utils(fatal_mod)

function new(string name = "fatal_mod");
  super.new(name);
endfunction

function action_e catch();
  string message;
  string id;

  if((get_id() == "red_id") & (get_severity() == UVM_FATAL)) begin
    set_message("Something went very wrong but was demoted to error");
    set_severity(UVM_ERROR);
  end
  return THROW;

endfunction

endclass
```

The following UVM code shows how these two call-backs are declared, constructed and then registered. The message_mod call back is registered only against the env.green component, so it will only be called when an object in the env.green generates a message. The fatal_demoter call-back has its add() method type argument set to null, which means that it applies to all messages generated in the UVM testbench.

```
// A test class that uses them:
class message_test extends uvm_component;

message_env env;
message_mod mess_mod;
fatal_mod fatal_demoter;

function void build_phase(uvm_phase phase);
  env = message_env::type_id::create("env", this);
  mess_mod = new("mess_mod");
  fatal_demoter = new("fatal_demoter");
  // The Message_mod report catcher is only applied to the env.green
component:
  uvm_report_cb::add(env.green, mess_mod);
  // The fatal_mod report catcher is applied to all component messaging
  uvm_report_cb::add(null, fatal_demoter);
```

```
endfunction
```

At the end of the UVM simulation, the report server will generate a summary of all the fatal, error and warning messages that had their severity demoted using the report catcher.

Testing Message Status

At the end of a UVM simulation, the report server issues a messaging summary to the transcript of the simulation. This will detail the number of each type of message severity that has been generated by the messaging system. If the simulation has been run with no report modifications, then this summary can be parsed and used as part of determining whether the simulation passed or not. For instance, the test case might be deemed to have passed if the scoreboard issues a pass message and there are no UVM_ERRORS or UVM_WARNINGS.

However, there may be more complicated scenarios where messages have been downgraded, or a specific error should have been provoked a certain number of times during the test case. In this situation the report server can be queried during the report phase to determine whether the test behavior is as expected.

The report server has two methods that are useful for this:

```
function int get_id_count(string id);
function int get_severity_count(uvm_severity severity);
```

The following code is an example of how these API calls might help to determine whether a test has passed or failed:

```
    function void report_phase(uvm_phase phase);
      uvm_coreservice_t cs;
      uvm_report_server svr;
      cs = uvm_coreservice_t::get();
      svr = cs.get_report_server();

      if (svr.get_severity_count(UVM_FATAL) == 0 &&
          ((svr.get_id_count("ASSERT_PARITY_ERROR") == 5) &&
(svr.get_severity_count(UVM_ERROR) == 5))) begin
         $write("** UVM TEST PASSED **\n");
      end
      else begin
         $write("!! UVM TEST FAILED !!\n");
      end
    endfunction
```

In this example, the test is expected to provoke 5 parity errors, therefore if there not 5 UVM_ERROR messages reported with the "ASSERT_PARITY_ERROR" id, then something has gone wrong in the test.

Note: - This approach to determining whether a test has passed or not is only relevant if all the messages generated in the UVM testbench code have used the UVM messaging system. It should be used in combination with other checks, like checking the status of a scoreboard. For instance, you could have a scenario where there are no UVM_ERRORs reported, but the test has failed because no transactions were sent to the DUT. Also, if assertions are being used, it would be unusual for those to be using the UVM messaging system, so you typically need an alternative way of checking for any assertion errors.

Command-Line Verbosity Control

There are several UVM plusargs that can be used to control messaging verbosity, actions and severity from the command line:

PlusArg	Description
+UVM_VERBOSITY=<uvm_verbosity>	This will set the default verbosity of all the components created in the UVM testbench to the specified verbosity.
+uvm_set_verbosity=<component>,<id>,<verbosity>,<phase>	This will set the verbosity of a specific component for a given UVM phase
+uvm_set_verbosity=<component>,<id>,<verbosity>,time,<time>	This will set the verbosity of a specific component from a specific time in the simulation.
+uvm_set_action=<component>,<id>,<severity><action>	This will set a messaging action against a component, id and severity
+uvm_set_severity=<component>,<id>,<current_severity>,<new_severity>	This will change the severity of a message against component and id.

Messaging in Sequences

Sequences typically use messaging, either for debug, traceability or to report on the outcome of a built-in checking mechanism.

A sequence object is derived from a uvm_object and therefore does not have the UVM messaging infrastructure built into it like a component object. However, for sequences and sequence_items the sequencer is used as the component from which the message is reported. This changes the way in which the sequence "path" is reported in the UVM messaging. For a sequence, the message path changes to the UVM hierarchical component path to the sequencer on which it is running, followed by a double at sign (@@) followed by the name of the sequence:

```
 UVM_INFO ../ef_sfr_test_pkg/sfr_test_seq.svh(12) @ 0:
uvm_test_top.env.agent.sequencer@@actual_seq [SFR_TEST_SEQ_START]
Starting test
```

If you are working with hierarchical sequences and you are interested in seeing where the sequence fits in the sequence hierarchy, then you need to ensure that whenever you call a sequence's start() method, you assign the handle to the parent sequence to the second argument of the call. If you don't do this, then a null handle is passed by default which means that only the leaf name of the sequence is reported. See line 6 in the body method of the sfr_test_upper_seq sequence in the code excerpt below for an example of how to call a sub-sequence start() method with the second, parent, argument assigned to this, the parent sequence.

```
class sfr_test_seq extends uvm_sequence #(sfr_seq_item);

task body;
  sfr_seq_item item = sfr_seq_item::type_id::create("item");

  `uvm_info("SFR_TEST_SEQ_START", "Starting test", UVM_MEDIUM)
  // Sequence action
  `uvm_info("SFR_TEST_SEQ_END", "Test completed", UVM_MEDIUM)

endtask: body
endclass: sfr_test_seq
```

```
0:class sfr_test_upper_seq extends uvm_sequence #(sfr_seq_item);
1:
2:task body;
3:  sfr_test_seq actual_seq =
sfr_test_seq::type_id::create("actual_seq");
4:
5:  // Using the second argument ensures that the parent sequence
appears in the sequence info messages
6:  actual_seq.start(m_sequencer, this);
7:endtask
8:endclass

// From the test:
task sfr_test::run_phase(uvm_phase phase);
  sfr_test_upper_seq seq =
sfr_test_upper_seq::type_id::create("test_seq");

  phase.raise_objection(this);
  seq.start(env.agent.sequencer);
  phase.drop_objection(this);

endtask: run_phase
```

If you follow this approach, then the resultant UVM_INFO message changes to this:

```
 UVM_INFO ../ef_sfr_test_pkg/sfr_test_seq.svh(12) @ 0:
uvm_test_top.env.agent.sequencer@@test_seq.actual_seq
[SFR_TEST_SEQ_START] Starting test
```